

Document: Flutter Developer Interview - Development Task

Product Area: KxInspections - Student App - Contesting Charges

Version: 1.0

Date Issued: 10 July 2026

Owner: Kinetic Software

Flutter Developer Interview: Contesting Charges Module

A practical, real-world Flutter development task designed to give candidates a chance to demonstrate their skills across UI, state management, testing, and multi-platform delivery.

1. Introduction

This task asks you to design and build a self-contained "Contesting Charges" module for the Student App portion of KxInspections. The module lets a student review a maintenance charge raised against their booking, view the supporting evidence from the inspection it was raised under, and either accept or contest the charge. It should reflect how the workflow would appear in the real Kinetic student-facing product.

The task is deliberately open-ended. The screenshots in the appendix set the baseline functionality and visual direction, but you are encouraged to extend the solution wherever you can add value - additional features, polish, animations, offline behaviour, richer state handling, and above all a solid automated test suite.

You are expected to use whatever tools help you deliver a high-quality result - IDE tooling, AI assistants, code generators, design tools and third-party packages are all fair game. We care about what you ship, how you structured it, and how you reason about it.

2. Context - Kinetic and KxInspections

Kinetic Software builds property management software for the Purpose-Built Student Accommodation (PBSA), Build-to-Rent and Higher Education sectors. KxInspections is our inspections product - it lets operations teams carry out property inspections, log damage and maintenance actions, and raise charges against a tenant's booking where appropriate.

The Student App is the tenant-facing side of that product. Today, students can already view their booking, raise maintenance tasks and see the outcome of inspections. What is missing is a first-class flow for the moment a charge is raised: seeing what the charge is for, reviewing the supporting evidence, and either accepting or contesting it. That is the flow you are being asked to build.

3. Task Overview

Build a Flutter application that implements the Contesting Charges workflow shown in the screenshots in the appendix. The build must run as:

- **Web:** A web app (Flutter Web).
- **Mobile:** A native mobile app on either Android or iOS (both is even better).

You should also provide:

- A working automated test suite (unit, widget and, where appropriate, integration tests).
- A short README that explains how to run each build target and how to run the tests.
- A mock data layer that simulates the backend so the app can be reviewed end-to-end without a live server.

4. Functional Requirements

The module has five key user-facing states. Each screenshot in the appendix corresponds to one of the states below. Treat the screenshots as the design intent - matching the layout and hierarchy is expected, but you may refine details where you see a clear improvement.

4.1 Maintenance Hub

This is the landing screen when the student opens the Maintenance area of the app. It brings together inventory, inspections, open tasks and charges for the current booking.

- Header with a back navigation control and the screen title "Maintenance Hub".
- Two tabs at the top: "Open" (default) and "History".
- A booking selector that shows the current booking's date range and property code (e.g. "MM/DD/YYYY > MM/DD/YYYY - OVA111") and lets the user switch bookings if more than one is available.
- A contextual notification banner near the top when there is an outstanding charge - showing the deadline and a "View" action that navigates to the Charge screen.
- An "Inventory" section listing the student's inventory with a status pill (e.g. Completed) and a "View report" link.
- An "Inspections" section listing any inspections associated with the booking (e.g. Pre-Arrival, Post-Arrival) with a status pill and a tap target that navigates to the View Inspection screen.
- An "Open Tasks" section with a prominent "Raise Task" button and a list of task cards. Task cards show a task ID ("#TCK57"), a category flag, notes, location, date and a status indicator (New / In Progress / Outstanding / Accepted / etc.).
- Charge cards should be visually distinct and always show the outstanding amount.
- A bottom navigation bar (Home, Book, Maintenance, Notifications). Only the Maintenance tab needs to be functional for this task, but the bar must render correctly.

State variations shown in the screenshots: (a) charge outstanding - blue banner with "View" and a red "Outstanding" status on the charge card; (b) charge accepted - green banner with "Pay" and a green

"Accepted" status on the charge card. Both variations must be supported.

4.2 View Inspection

Opened from the Inspections list on the Maintenance Hub. Shows what happened during a specific inspection and any charges or actions that came out of it.

- Screen title in the format "[Inspection type] Inspection - [Property code] ([Room])" (e.g. "Pre-Arrival Inspection - OVA111 (Ensuite)").
- Intro paragraph: "Below are the actions and updates created as part of this inspection."
- "Outstanding Charges" section listing any charges raised - each card shows the type (Replace / Repair / Clean), the item, notes, location, amount, and a status.
- "General" section for free-text notes from the inspector, each date-stamped.
- "Item Actions" section listing items that require action, with a thumbnail image, item name, notes and charge amount.
- "Item Updates" section listing items that were inspected and did not require action, with a short condition note.
- Tapping an item with a charge on it should navigate to the Charge screen.

4.3 View Charge

Reached from the notification banner on the Maintenance Hub, a charge card, or an item on the View Inspection screen. This is the moment of decision for the student.

- Screen title "Charge" with a back button.
- Prominent intro paragraph that explains what will happen, including the deadline and the default behaviour if no action is taken ("...or this will automatically accepted on your behalf").
- Read-only fields: Item, Type (Replace / Repair / Clean), Notes, Amount.
- Photo evidence: a horizontally-scrolling row of image thumbnails; tapping a thumbnail opens a full-screen image viewer (pinch to zoom on mobile, click-to-zoom on web).
- Two primary actions at the bottom: a secondary "Contest" button (outlined, red) and a primary "Accept" button (filled, navy).
- Accepting a charge must move the charge to the "Accepted" state, dismiss the outstanding notification banner, and surface a success confirmation.
- Contesting a charge must navigate to the Contest Charge screen.

4.4 Contest Charge

The form the student fills in when they don't agree with a charge.

- Screen title "Contest Charge" with a back button.
- Instruction copy: "Please explain in your own words why you wish to contest this charge."
- A multi-line "Reason" text field with an edit affordance. The field must handle long, wrapped text gracefully.
- A "Submit" action, disabled until the reason meets a minimum length (e.g. 10 characters).
- Submission should optimistically move the charge to a "Contested" state, dismiss the outstanding notification banner and surface a confirmation. The submitted reason should be retrievable from the History tab.

4.5 Charge Lifecycle and Business Rules

The following states and transitions should be modelled explicitly in the code, not just at the UI layer.

State	Enters when...	Notification banner
Outstanding	Charge raised against the student's account	Blue - "A charge has been added to your account. Please review and pay this by [date]."
Accepted	Student taps Accept, or the deadline passes without action	Green - "Thank you for accepting this charge. Please go to your statement to pay."
Contested	Student submits the Contest Charge form	Amber (your choice) - "Your contest has been submitted. We will review it and get back to you."
Resolved / Paid	(Bonus) Charge fully paid or resolved by the operator	None (moved to History tab)

- The deadline countdown is derived from the charge's "raised on" date plus a configurable grace period (default 30 days).
- History tab shows all charges that are no longer Outstanding, most recent first, with the final state and the reason supplied (if contested).

5. Technical Requirements

5.1 Platform and Tooling

- **Language:** Flutter (latest stable channel).
- **Targets - required:** Web build (deployed or runnable via `flutter run -d chrome`).
- **Targets - required:** Android or iOS (either preferred). Provide a runnable build or a signed debug APK/IPA.

5.2 Architecture

There is no single correct architecture. We are looking for the choices to be intentional and consistent. Whatever you pick, please make sure the following are true:

- The UI layer is thin. Business logic and state transitions live outside widgets.
- State management is explicit (Riverpod, Bloc, Provider, Redux, MobX - your choice) and rationalised in the README.
- Data access sits behind a repository/service interface so mock and real implementations can be swapped without touching the UI.
- Navigation is centralised (e.g. `go_router` or `auto_route`) rather than ad-hoc Navigator calls.
- Domain models are strongly typed (ideally with `freezed` or equivalent) and easy to serialise.

5.3 Data and Persistence

- Provide seed data (bookings, inspections, tasks, charges) as JSON fixtures.
- The app must run end-to-end against these fixtures with no live backend required.
- Local persistence of user actions (accepted, contested) using `shared_preferences`, `Hive`, `drift` or similar, so the app survives a reload.
- Bonus: a mock HTTP layer (e.g. using `dio` + interceptors, or a local `FakeApi`) that mimics network latency and error responses, and can be flipped between mock and real via a configuration flag.

5.4 Automated Testing (Required)

A working automated test suite is a hard requirement, not a bonus. All tests must run with a single command (documented in the README). Aim for meaningful coverage of behaviour, not coverage of trivial getters.

- **Unit tests:** Cover charge state transitions, deadline handling and validation logic.
- **Widget tests:** Cover key widgets: Maintenance Hub, View Charge and Contest Charge. Use `flutter_test` and pump multiple states.
- **Golden tests:** One or more golden tests (`matchesGoldenFile`) for the notification banners and charge card variants.
- **Integration test:** At least one end-to-end flow: view charge -> contest -> submit -> back on Maintenance Hub with updated state. `integration_test` package or `patrol` are fine.
- **CI:** A GitHub Actions (or equivalent) workflow that runs the test suite on push. Bonus if it also builds the web target and uploads the artefact.

5.5 Non-Functional Requirements

- **Responsive layout:** Layouts must be responsive from ~360px mobile up to a wide desktop viewport. No horizontal scrolling on either.
- **Accessibility:** Meet WCAG AA colour contrast, add semantic labels on interactive elements, and ensure the flow is fully keyboard-navigable on web.
- **Performance:** Sensible use of `const` constructors, minimal rebuilds, images cached appropriately (e.g. `cached_network_image`).
- **Localisation-ready:** All user-facing strings should sit in a single place so the app is ready for future localisation. Providing a second locale (en / cy is a nice touch) is a bonus.

6. Bonus Features - Room to Impress

The following are not required. They are opportunities to differentiate your submission. Feel free to add anything else you think would improve the module.

Idea	Why it matters
Dark mode + system theme awareness	Modern default for a student-facing app.
Push notifications (local, or via FCM) when a charge is raised	Reflects the real-world trigger for this flow.
Offline mode with an outbox pattern for Accept / Contest actions	Students may be on flaky wifi in halls; queuing actions is realistic.
Rich attachments on the contest form (photos, video, PDFs)	Mirrors how disputes actually get resolved.
A charge "statement" view listing multiple outstanding charges with a running total, plus a mock payment step	Extends the flow all the way to resolution.
Analytics event tracking (mock) for each state transition	Shows product-thinking.
Animations on state transitions (banner swap, card pill colour change)	Adds polish that a student user would actually notice.
Deep-linking (e.g. <code>/charges/{id}</code>) on	Demonstrates <code>go_router</code> / deep linking know-how.

the web target	
Accessibility audit output and screen-reader walkthrough video	Concrete evidence of thoughtful accessibility work.
A design token layer (colours, spacing, typography) usable across screens	Sets you up to scale beyond one module.

7. Evaluation Criteria

Submissions will be scored against the criteria below. The weighting is a guide, not a strict formula - a submission that is exceptional in one area can offset a weaker one.

Area	Weight	What we look for
Functional completeness	25%	The five screens, the state transitions, and the described business rules all work end-to-end without a live backend.
Code quality and architecture	20%	Readable, well-organised code. Sensible state management. Clear separation of concerns. No dead code, TODOs left in, or debug prints shipped.
UI quality and polish	20%	Faithful to the screenshots at the same time as feeling like a real product. Responsive, accessible, well-typed. Nice touches.
Automated testing	15%	Meaningful unit, widget and integration coverage. Tests actually assert behaviour. All tests pass on a fresh checkout.
Additional features	10%	Bonus features from Section 6 or your own ideas. Quality over quantity.
Documentation and delivery	10%	Clear README. Easy setup. Runnable web + mobile artefacts. Honest notes on any known gaps.

8. Deliverables

- **Source code:** A public Git repository (GitHub, GitLab, Bitbucket) with the full source. Include a commit history that tells the story of how you built it.
- **Web build:** A deployed web build (Firebase Hosting, GitHub Pages, Netlify, Vercel - any is fine) OR clear `flutter run -d chrome` instructions.
- **Mobile build:** A debug APK, an IPA (or TestFlight link), or clear `flutter run` instructions for at least one of Android / iOS.
- **README:** Setup, run and test instructions. Architecture overview. Decisions and trade-offs. Known gaps.
- **Test suite:** A single command that runs the whole suite (e.g. `flutter test`) and a note on how to view coverage.
- **Walkthrough (optional but recommended):** A ~3 minute Loom / screen recording walking through the app on web and mobile, and calling out any features you're proud of. Not required, but strongly encouraged.

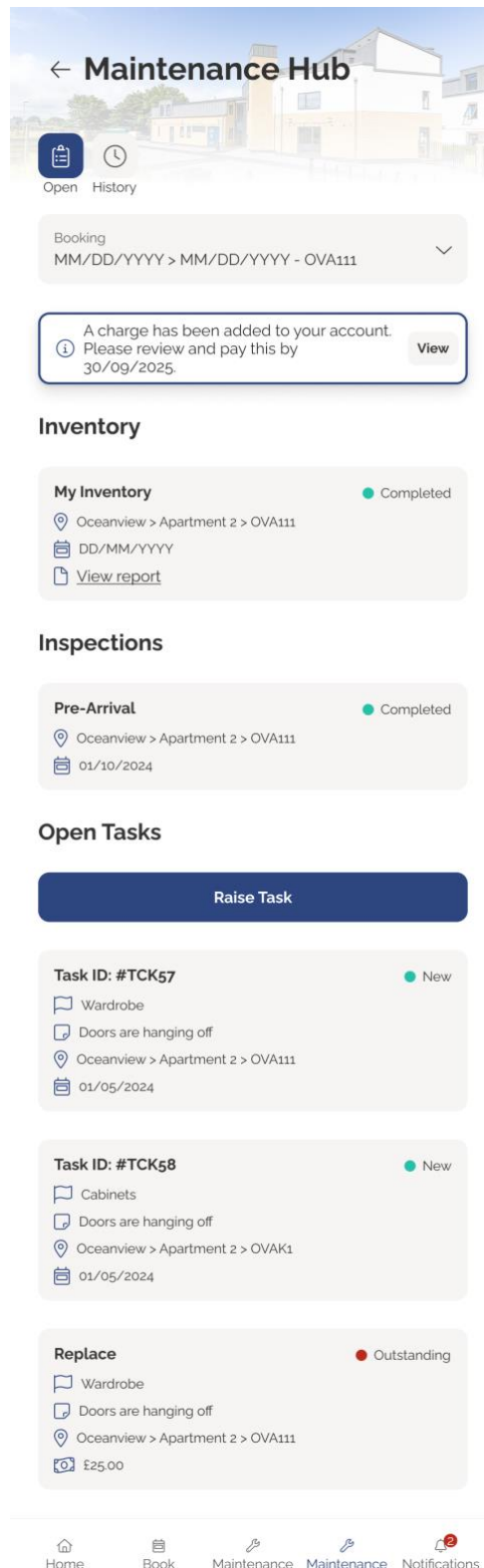
9. Timeline and Submission

- **Time budget:** You have 7 calendar days from the date this brief is issued to submit.
- **Effort:** We're not measuring hands-on-keyboard hours. Use your time as you see fit.
- **Submitting:** Reply to the email that sent you this brief with the Git repo URL, any deployed build URLs, and (if applicable) the walkthrough video link.
- **Questions:** Ask. There are no penalties for asking, and no bonus points for guessing.

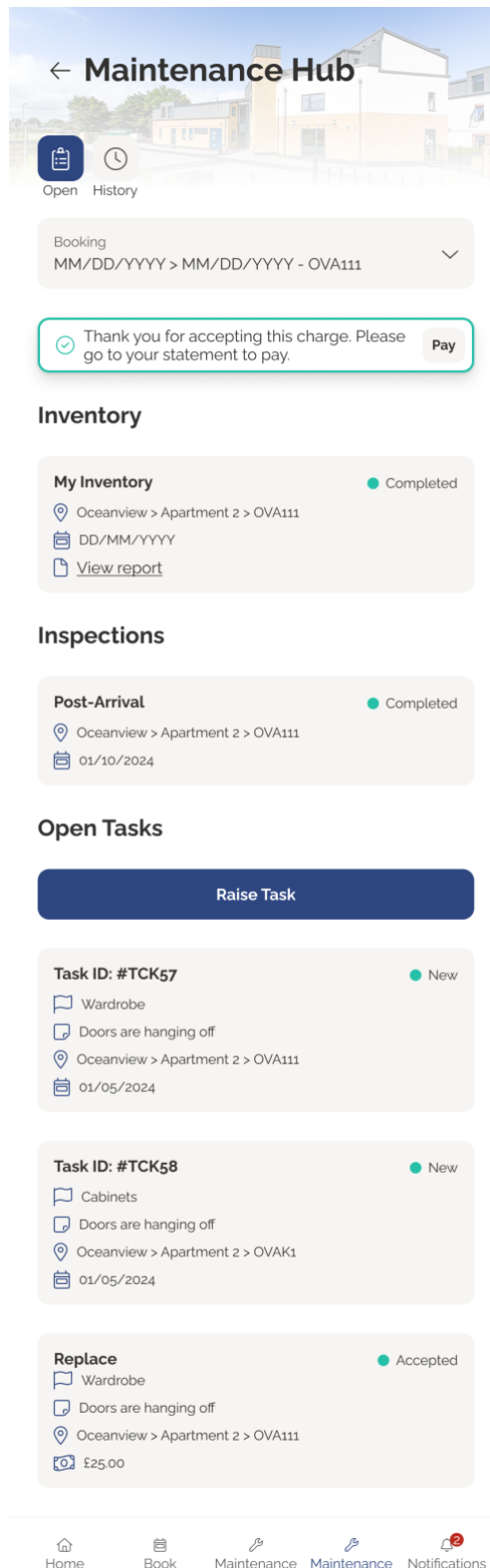
Use anything that helps you ship: IDE tooling, GitHub Copilot, Claude, ChatGPT, Cursor, design tools, template repos, published Flutter packages. Please be transparent about it in your README. We want to see what you deliver with the tools available to a modern engineer, not test whether you can build Flutter from memory.

10. Appendix - Reference Screenshots

The five screenshots below define the visual direction. They are mocked from the Kinetic design system and represent how the module should feel when finished. Refine details as you see fit - copy the intent, not the pixels.



Screenshot 1 - Maintenance Hub with an outstanding charge notification banner (blue, "View" action).



Screenshot 2 - Maintenance Hub after the charge has been accepted (green banner, "Pay" action, charge card now Accepted).



Below are the actions and updates created as part of this inspection.

Outstanding Charges

Replace

● Outstanding

🚪

Wardrobe

🚪

Doors are hanging off

📍

Oceanview > Apartment 2 > OVA111

💰

£25.00

General

10/09/24

This room needs to be cleaned please - will re-inspect in 1 week.

Item Actions



Item

Notes

Charge

Wardrobe

Doors are hanging off.

£25.00

Item Updates

Bed

No visible signs of damage.

Lamp

Slight wear and tear.

Screenshot 3 - View Inspection showing outstanding charges, general notes, item actions and item updates.



The below charge has been raised against your account. Please accept or contest this by 30/09/2025 or this will automatically accepted on your behalf.

Item
Wardrobe

Type
Replace

Notes
Doors are hanging off

Amount
£25.00



Contest

Accept

Screenshot 4 - View Charge with item details, photo evidence, and the Accept / Contest actions.



Please explain in your own words why you wish to contest this charge.

Reason

This wasn't me this was my roommate. I was on holiday



Submit

Screenshot 5 - Contest Charge form with the reason text field and the Submit action.